Russo Giovanni

Promo 2026 – MSc Computer Science, Digital Security track

Outsight

Sophia Antipolis, Valbonne

Dates of Internship: 16 February 2026 – 31 July 2026

Migrating Legacy Services to a Cloud-Native Architecture: A Case Study on Outsight Cloud

EURECOM advisor:
Prof. Adlen Ksentini,

Company advisor:
Kasper Rutten,
Anatolii Kostrygin

Confidential thesis report

YES ☒ *NO* ☐

EURECOM

EURECOM
Sophia Antipolis

DECLARATION POUR LE RAPPORT DE STAGE
DECLARATION FOR THE MASTER'S THESIS

Je garantis que le rapport est mon travail original et que je n'ai pas reçu d'aide extérieure. Seules les sources citées ont été utilisées dans ce projet. Les parties qui sont des citations directes ou des paraphrases sont identifiées comme telles.

I warrant, that the thesis is my original work and that I have not received outside assistance. Only the sources cited have been used in this report. Parts that are direct quotes or paraphrases are identified as such.

À Biot, *in Biot*
Date : .....28/07/2026.....

Nom Prénom : Giovanni Russo
Name First Name

Signature :

Abstracts (English & French)

Abstract

This report investigates the migration of Outsight Cloud, a production platform for LiDAR-related workflows and assets, from a legacy service-oriented architecture toward a cloud-native environment on AWS. The original platform ran on DigitalOcean virtual machines orchestrated with Docker Compose, and relied on Airtable as its primary operational datastore, an arrangement that progressively introduced scalability, consistency, and operational-governance limitations. The work proposes and evaluates an incremental migration strategy based on Infrastructure as Code, managed cloud services, and controlled coexistence between legacy and target components. Particular attention is given to the transition from Airtable to PostgreSQL, the move from VM-based deployments to ECS Fargate, Terraform-driven provisioning, and validation mechanisms that reduce migration risk while preserving service continuity. The evaluation shows measurable improvements in infrastructure cost, end-to-end latency, and operational effort, illustrating how cloud-native modernization can be achieved incrementally in a real industrial context.

Résumé

Ce mémoire étudie la migration d'Outsight Cloud, une plateforme de production dédiée aux traitements et aux ressources liés au LiDAR, d'une architecture orientée services héritée vers un environnement cloud-native sur AWS. La plateforme initiale reposait sur des machines virtuelles DigitalOcean orchestrées avec Docker Compose et utilisait Airtable comme base de données opérationnelle principale, ce qui a progressivement engendré des limites de passage à l'échelle, de cohérence et de gouvernance opérationnelle. Le travail propose et évalue une stratégie de migration incrémentale fondée sur l'Infrastructure as Code, des services cloud managés et une coexistence contrôlée entre composants hérités et cibles. Une attention particulière est portée au passage d'Airtable à PostgreSQL, à la migration vers ECS Fargate, au provisionnement piloté par Terraform et aux mécanismes de validation réduisant le risque tout en préservant la continuité de service. L'évaluation démontre des gains mesurables en coût d'infrastructure, en latence et en effort opérationnel, illustrant une modernisation cloud-native progressive dans un contexte industriel réel.

Acknowledgements

I would like to thank Outsight for the opportunity to carry out this internship, and my company and academic supervisors for their guidance and support throughout the project. I am also grateful to EURECOM and to Politecnico di Torino for the academic foundation that made this work possible, and to my family and friends for their constant support.

Contents

Plagiarism Declaration I

Abstracts (English & French) II

Acknowledgements III

1 Introduction 1
- 1.1 General Presentation of the Internship 1
- 1.2 Industrial Context and Legacy Constraints 1
- 1.3 Report Structure 2

2 The Company and the Business Sector 4
- 2.1 Outsight and Spatial AI 4
- 2.2 Customers and Use Cases 4
- 2.3 Outsight Cloud 5

3 Presentation of the Internship Topic 6

I Development (Technical Chapters) 8

4 Background and Related Work 9
- 4.1 Cloud-Native Principles for Legacy Modernization 9
  - 4.1.1 Incremental Delivery and Operability 9
  - 4.1.2 Infrastructure as Code 10
- 4.2 Migration Strategies and Patterns 10
  - 4.2.1 Cloud Migration Fundamentals 10

4.2.2 System Coexistence in Incremental Migration 11
4.2.3 Decision-Driven Migration 11
4.3 IAM Modernization in Legacy Systems 12
4.4 Limitations in Existing Approaches and Positioning 12

5 Case Study Baseline (As-Is System) 14
5.1 Current Architecture and Data Flow 14
5.2 Airtable as Primary Data Storage 15
5.3 Limitations of Airtable 16
5.4 Motivation for Migration to PostgreSQL 16

6 Migration Methodology and Decision Framework 18
6.1 Migration Goals and Success Criteria 18
6.2 Candidate Solutions Considered 19
6.2.1 Infrastructure and Runtime Options 19
6.2.2 Container Orchestration: Kubernetes vs ECS Fargate 19
6.2.3 Alternative Cloud Platforms Considered 20
6.2.4 Data-Layer and Identity Options 21
6.3 Technical and Economic Evaluation Criteria 21
6.4 Selected Strategy and Rationale 22
6.5 Risk Management and Coexistence Plan 22

7 Target Architecture and Solution Design (To-Be) 24
7.1 Architecture Overview 24
7.2 Client, Delivery, and Routing Layers 24
7.3 Application Layer 26
7.4 Managed Services Layer 26
7.5 Network and Security Topology 27
7.6 Architectural Benefits 27

8 Migration Implementation 29
8.1 Incremental Execution Plan 29
8.2 Infrastructure as Code and Environment Separation 30
8.2.1 Terraform-Driven Provisioning 31
8.3 Infrastructure Migration Execution 32

8.4 Data Migration Tooling and Validation 33

9 Evaluation and Discussion 34

9.1 Benchmark Methodology and Demand Profile 34

9.2 Cost and TCO Analysis 36

9.3 Latency and Performance Analysis 40

9.4 Operational Cost and Efficiency 43

10 Conclusion 46

10.1 Key Outcomes 46

10.2 Limitations and Open Questions 47

10.3 Future Work 47

Chapter 1

Introduction

1.1 General Presentation of the Internship

This report was prepared as part of a Master's internship carried out at **Outsight**, in **Sophia Antipolis, Valbonne**, France, between **16 February 2026 and 31 July 2026**. The internship was completed within Outsight's engineering team, and consisted in analyzing and carrying out the migration of Outsight Cloud (OC), the company's platform for managing LiDAR-related workflows, simulations, organizations, sites, and associated resources, from a legacy virtual-machine-based infrastructure toward a cloud-native environment on AWS. Chapter 2 presents the company and its business sector, Chapter 3 presents the internship topic in more detail, and Section 1.3 outlines the remainder of this report.

1.2 Industrial Context and Legacy Constraints

Cloud computing has transformed the way software systems are designed, deployed, and maintained, offering scalable infrastructure, managed services, and automated deployment capabilities that are difficult to achieve with self-managed environments. As a consequence, many organizations are progressively modernizing legacy applications toward cloud-native architectures.

Migrating an existing production platform to the cloud, however, is rarely straightforward. Legacy systems are typically the result of years of incremental development

shaped by evolving business requirements, and over time they accumulate architectural dependencies and operational practices that complicate scalability, maintainability, and long-term evolution.

This report analyzes the migration of Outsight Cloud (OC), a platform used to manage LiDAR-related workflows, simulations, organizations, sites, and associated resources. Before the migration, the platform relied on a virtual-machine-based infrastructure hosted on DigitalOcean Droplets and orchestrated through Docker Compose, integrating several external services, most notably Airtable for core data management. This architecture enabled rapid development during the early product lifecycle, but as usage and integrations expanded it began to show clear signs of architectural strain.

1.3 Report Structure

This report is organized as follows:

- **Chapter 2** presents the company, Outsight, and its business sector.
- **Chapter 3** presents the internship topic, i.e. the engineering problem addressed in this work.
- **Chapter 4** introduces the conceptual and related-work foundations on cloud-native modernization, migration patterns, and IAM evolution in legacy systems.
- **Chapter 5** describes the case-study baseline (as-is architecture), including technical debt and cost-related migration drivers.
- **Chapter 6** presents the migration methodology and decision framework, including candidate options and selection rationale.
- **Chapter 7** defines the target architecture (to-be), its main components, and design trade-offs.
- **Chapter 8** details the incremental migration implementation, with focus on IaC, data migration tooling, and the validation pipeline.

- **Chapter 9** discusses the evaluation approach and the evidence on migration cost, performance, and operational impact.
- **Chapter 10** concludes by summarizing contributions and outlining future work.

Chapter 2

The Company and the Business Sector

2.1 Outsight and Spatial AI

Outsight is a company operating in the **Spatial AI** sector: it develops solutions that track and digitize the motion of people and vehicles using 3D LiDAR data. By turning raw LiDAR point clouds into structured, real-time spatial information, Outsight enables its customers to understand how people and vehicles move through physical spaces without relying on cameras or other identity-revealing sensors.

2.2 Customers and Use Cases

Operators of transportation hubs such as airports and train stations, as well as sports venues, road infrastructure, and industrial sites, use Outsight's Spatial Intelligence data to improve their operations, deliver a better experience to visitors, and increase security and safety. Because the underlying LiDAR-based sensing approach is accurate while remaining anonymous, it is particularly well suited to environments where privacy-preserving monitoring of large flows of people and vehicles is required.

2.3 Outsight Cloud

Outsight Cloud (OC) is the company's software platform supporting this business: it manages the LiDAR-related workflows, simulations, organizations, sites, and associated resources needed to deploy and operate Outsight's Spatial Intelligence solutions at customer sites. Outsight Cloud is the platform whose migration from a legacy infrastructure to a cloud-native architecture on AWS is the subject of this report, and the internship on which it is based was carried out within the engineering team responsible for this platform.

Chapter 3

Presentation of the Internship Topic

The engineering problem addressed in this work concerns the migration of Outsight Cloud from a legacy architecture toward a cloud-native environment under real industrial and operational constraints.

Over time, the platform became strongly dependent on external services, especially Airtable, which was deeply embedded in application workflows. The absence of a fully relational data model limited consistency guarantees and complicated the management of relationships between entities, while operational procedures increasingly relied on manual workflows that were error-prone and hard to scale. In parallel, the original deployment model became progressively harder to scale and maintain.

To address these limitations, the company initiated a cloudification process aimed at redesigning the platform architecture and migrating toward a managed cloud environment. The selection of the target platform and strategy resulted from a structured evaluation of candidate solutions against technical, operational, and economic criteria. A central challenge was defining an execution sequence able to reduce technical and operational risk before final cutover, relying on validation workflows, reproducible infrastructure management, and controlled transition mechanisms. The migration was driven by a set of concrete objectives:

- Reduce operational costs by replacing Airtable and self-managed infrastructure with managed cloud services.

- Improve performance and data-access efficiency through PostgreSQL as the primary relational datastore.
- Reduce operational overhead by automating provisioning, deployment, and maintenance through Infrastructure as Code.
- Re-establish ownership of core data models and infrastructure definitions, reducing dependency on vendor-specific platforms.
- Minimize migration risk and service disruption through incremental coexistence, validation, and controlled transition.
- Establish a scalable and maintainable architectural foundation for long-term platform evolution.

Part I

Development (Technical Chapters)

Chapter 4

Background and Related Work

This chapter provides the conceptual background required to frame the migration problem and positions the report with respect to related work.

4.1 Cloud-Native Principles for Legacy Modernization

Cloud-native modernization is not limited to container adoption or workload relocation. In migration contexts, it also requires reproducible infrastructure, explicit service boundaries, deployment automation, observability, and validation mechanisms able to support incremental system evolution, all of which become critical when architectural changes affect production workflows and persistent data.

In *Building Microservices*, Newman frames modern service-oriented architectures around independently deployable services with explicit boundaries and hidden internal state [1]. This principle separates external contracts from internal implementation details, reducing the risk that changes in one component propagate unexpectedly across the platform.

4.1.1 Incremental Delivery and Operability

A recurring principle in modernization literature is the preference for incremental delivery over full-system replacement. Newman argues against big-bang rewrites, since

large simultaneous replacements concentrate risk and delay feedback [1]. Decomposing migration work into smaller steps makes it possible to start from lower-risk components, build confidence progressively, and validate changes before irreversible decisions. To be effective, incremental delivery requires infrastructure automation, environment consistency, reproducible deployment workflows, and testable interfaces.

4.1.2 Infrastructure as Code

Infrastructure as Code (IaC) is a key enabler for cloud-native modernization because it transforms infrastructure configuration into versioned and reproducible artifacts, replacing manual provisioning with declarative definitions managed through automated workflows. Newman identifies IaC as a core deployment principle because configuration stored in source control allows environments to be recreated deterministically [1]. Morris further identifies three core practices for effective IaC: defining everything as code, continuously testing and delivering all work in progress, and building small loosely coupled pieces [2]. Together, these practices improve traceability, reproducibility, and consistency, and in migration projects they simplify environment replication, deployment governance, and rollback management.

4.2 Migration Strategies and Patterns

Cloud migration projects can follow different strategies depending on system complexity, operational requirements, and modernization goals. In practice, enterprise migrations often combine multiple strategies across subsystems according to their coupling, criticality, effort, and risk.

4.2.1 Cloud Migration Fundamentals

Cloud migration is the process of moving applications, data, and infrastructure from traditional environments to cloud platforms, typically executed progressively through staged rollout and validation activities to minimize disruption [3]. Migration planning must evaluate not only where workloads will run, but also how infrastructure responsibilities, automation practices, and operational governance will change.

Reference guidance classifies migration strategies into three major categories [3]:

- **Rehosting**, which moves workloads to the cloud with minimal architectural changes.
- **Replatforming**, which introduces selective adaptations to better exploit cloud capabilities while preserving the overall application architecture.
- **Refactoring**, which redesigns parts of the system to align with cloud-native principles.

The Outsight Cloud migration is best described as a hybrid strategy: primarily a replatforming effort (transition from DigitalOcean-hosted workloads to AWS managed services and ECS Fargate) complemented by selective refactoring (the transition from Airtable to PostgreSQL, the introduction of Terraform, and adaptations to deployment and routing workflows).

4.2.2 System Coexistence in Incremental Migration

For production systems that cannot tolerate interruption, coexistence between legacy and target components becomes a critical requirement: both systems frequently need to operate simultaneously while compatibility is progressively validated. Newman describes the *strangler fig pattern* for wrapping legacy systems incrementally and redirecting calls to new implementations, and *parallel-run* approaches where old and new implementations execute side by side and their results are compared before cutover [1]. Morris describes the *expand and contract* pattern for changing live infrastructure without disruption: new resources are added alongside existing ones, consumers migrate incrementally, and only then are legacy components removed [2]. This coexistence model introduces additional complexity (contract compatibility, dual-backend validation, rollback, progressive switchover) and makes migration validation an integral part of the architecture rather than a secondary activity.

4.2.3 Decision-Driven Migration

Related work frequently describes migration patterns from a purely technological perspective. In industrial contexts, however, migration decisions must also consider

operational constraints, organizational impact, recurring costs, maintainability, and risk. Migration strategies should therefore be supported by explicit decision frameworks combining technical and economic rationale, particularly where architectural choices affect both infrastructure sustainability and long-term governance.

4.3 IAM Modernization in Legacy Systems

Identity and Access Management (IAM) modernization is a recurrent challenge in legacy platforms, which historically implemented authentication and authorization logic directly inside application workflows, generating tightly coupled models that are difficult to evolve. In AWS-based environments, *AWS IAM* governs access to cloud resources by associating principals (users, roles, federated identities, or applications) with permission policies that determine which operations are allowed [4]. Modern cloud-native architectures instead rely on standardized protocols such as OAuth 2.0 and OpenID Connect and dedicated identity services to improve interoperability and security. Migrating toward standards-based IAM requires decoupling identity management from application-specific implementations while preserving compatibility with existing workflows. The choice between open-source and managed IAM solutions also affects operational complexity: managed services reduce infrastructure responsibilities and provide built-in lifecycle management, whereas open-source solutions offer greater customization at the cost of higher operational overhead.

4.4 Limitations in Existing Approaches and Positioning

The literature provides strong conceptual foundations on migration patterns and IAM technologies, but industrial guidance often remains fragmented. Technical architecture decisions are frequently discussed without detailed consideration of operational costs and long-term maintenance, and many case studies focus on final target architectures while providing limited evidence on coexistence management and intermediate validation. Morris emphasizes that continuously testing infrastructure

changes as they are developed is essential for catching errors early [2]; such validation practices (dual-backend testing, data-consistency verification, environment-separated workflows) are essential for reducing migration risk during long transitions.

This report addresses these limitations through a decision-oriented migration case study focused on incremental architectural evolution under real industrial constraints. Rather than merely describing a target AWS architecture, it analyzes the reasoning process connecting baseline limitations, migration decisions, infrastructure automation, data-layer transition, and validation workflows. Although centered on Outsight Cloud, the principles and practices discussed are applicable to broader legacy modernization scenarios involving progressive cloud-native adoption.

Chapter 5

Case Study Baseline (As-Is System)

This chapter describes the initial state of Outsight Cloud before the migration strategy was formalized, establishing the architectural and operational baseline against which migration decisions are evaluated.

5.1 Current Architecture and Data Flow

Before the migration to AWS, Outsight Cloud relied on a containerized architecture designed for rapid feature delivery. The platform managed LiDAR-related workflows and assets through a web application and backend API stack composed of several interconnected components: a **Vue 3 SPA frontend**; a **Node.js/Express backend API** implementing business logic and integrations; **Redis** for session management; **Docker Compose** orchestrating containers on **DigitalOcean Droplets**; a **Traefik** reverse proxy for HTTPS termination and routing; **Airtable** as the primary operational datastore for users, organizations, sites, simulations, and permissions; **S3-compatible object storage** for simulation files; **GitBook** for documentation; and **Slack and Customer.io** integrations for notifications.

From an infrastructure perspective, the architecture followed a virtual-machine-centric model in which scaling, upgrades, deployment, and maintenance were handled directly by the engineering team. While sufficient during the early stages, this

approach increasingly exposed limitations related to scalability, operational governance, fault tolerance, and infrastructure reproducibility.

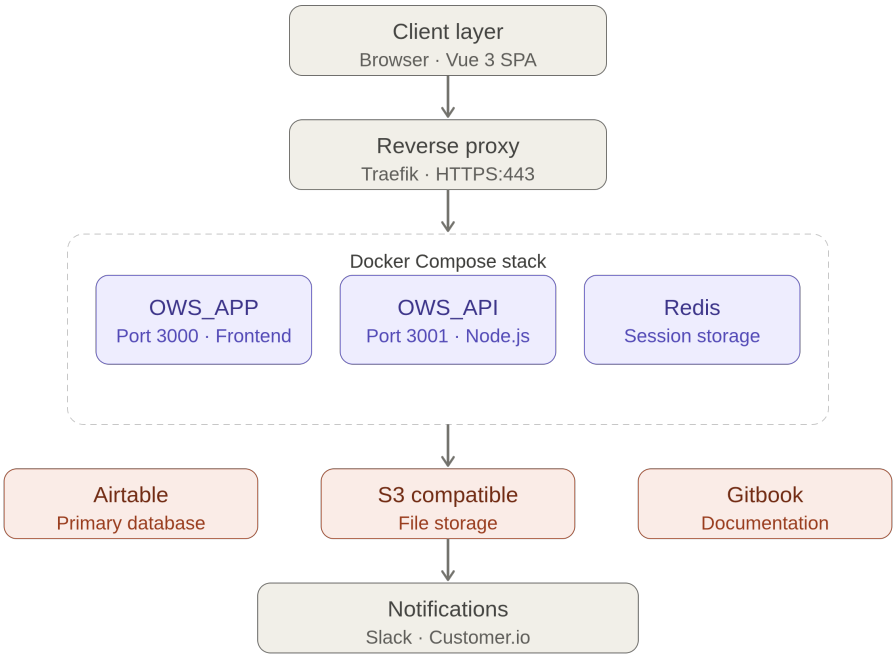

Figure 5.1. Outsight Cloud baseline architecture before the migration.

5.2 Airtable as Primary Data Storage

Airtable was more than a persistence layer: it acted simultaneously as operational datastore, workflow-management layer, and integration hub. Airtable is a cloud-based platform combining spreadsheet-like interfaces with lightweight database capabilities, organizing data into bases, tables, records, and fields, and providing views, forms, automations, and API access [5]. In Outsight Cloud these capabilities were exercised through API access, operational forms, and integrations such as Slack-based notifications. During early phases, Airtable's accessibility for non-technical

stakeholders accelerated development and simplified collaboration between engineering and operations.

5.3 Limitations of Airtable

As Outsight Cloud grew, the limitations of using Airtable as primary backend infrastructure became increasingly evident, affecting data consistency, maintainability, operational workflows, and scalability.

Data modeling and consistency. Relationships between entities such as organizations, sites, simulations, and permissions were represented through linked records and lookup fields rather than explicit relational constraints. Consistency guarantees therefore depended on application-side logic instead of database-level enforcement, and lookup fields sometimes exposed heterogeneous structures (scalar values or arrays depending on context), increasing backend complexity and technical debt.

Limited relational capabilities. Although Airtable supports basic linked-record relationships, it does not provide the full relational capabilities of SQL systems such as PostgreSQL. Advanced joins, indexing, transactional consistency, and complex filtering were significantly more limited, requiring growing amounts of application-side logic to compensate for missing relational features.

Operational inefficiency and cost. Several workflows, such as user onboarding and account management, relied on manual interactions through Airtable forms, slowing processes and increasing the risk of error as the number of users and simulations grew. Over time Airtable evolved from a simple datastore into a critical architectural dependency tightly coupled with application workflows, reducing flexibility. The recurring per-seat cost also became significant, creating additional pressure to adopt a more scalable and controllable backend.

5.4 Motivation for Migration to PostgreSQL

The migration from Airtable to PostgreSQL was initiated to address these architectural, operational, and economic limitations. The objective was not merely to replace one storage technology with another, but to transition from a semi-structured

operational backend toward a maintainable and scalable relational architecture. PostgreSQL introduced explicit relational schemas based on primary and foreign keys, stronger consistency and integrity guarantees, advanced querying, transactional support, and indexing, simplifying backend implementation. From an architectural perspective, the migration reduced dependence on Airtable-specific structures and allowed the platform to regain ownership of its data model and persistence logic.

Chapter 6

Migration Methodology and Decision Framework

This chapter presents the migration methodology adopted for Outsight Cloud and the decision process that guided the selection of the target architecture and strategy, considering not only the technologies adopted but also the technical, operational, and economic factors involved.

6.1 Migration Goals and Success Criteria

Before defining the target architecture, a set of migration goals was identified to establish a structured decision framework. The first objective was to avoid disruptive replacement or prolonged service interruptions, which translated into concrete success criteria: staged rollout capability, rollback readiness, validation of critical workflows before cutover, and separation between development and production activities. Further objectives were reducing architectural coupling to legacy dependencies (particularly Airtable), improving scalability and maintainability through managed services and reproducible infrastructure, and ensuring deployment reproducibility and environment governance. In addition to technical goals, the process required explicit evaluation of operational sustainability and recurring costs, so architectural decisions had to balance technical quality, implementation complexity, and long-term maintainability.

6.2 Candidate Solutions Considered

The design process considered multiple alternatives for each architectural dimension, focusing on solutions able to balance migration risk, production stability, maintainability, and implementation effort rather than adopting a purely technology-driven approach.

6.2.1 Infrastructure and Runtime Options

Three infrastructure strategies were considered. The first preserved the existing deployment model while hardening operational workflows; this minimized short-term complexity but did not address scalability or governance concerns. The second migrated toward managed cloud services combined with IaC, introducing a structured cloud-native operational model while preserving incremental migration. The third adopted a fully orchestration-oriented redesign (e.g. Kubernetes) from the start, which provided strong scalability but significantly increased complexity and operational overhead during early transition.

6.2.2 Container Orchestration: Kubernetes vs ECS Fargate

Kubernetes is a portable, extensible open-source platform for managing containerized workloads through declarative configuration, providing scaling, failover, service discovery, load balancing, and controlled rollouts [6]. However, adopting Kubernetes would have introduced operational complexity (cluster administration, networking, upgrades, monitoring, capacity planning, security) not justified by the requirements of Outsight Cloud at the time. This is consistent with Newman's caution against premature platform complexity: small service landscapes do not necessarily require Kubernetes-level orchestration when cluster-management cost would distract from migration goals [1].

In contrast, Amazon ECS Fargate provides a managed container execution environment that eliminates the need to provision and maintain worker nodes, with native integration with ALB, CloudWatch, IAM, and Auto Scaling while preserving support for Docker-based workloads. It therefore enabled a smoother transition from the legacy Docker Compose model, and offered the best balance between scalability and operational simplicity for an incremental migration. Kubernetes remains

a viable option for future evolution.

6.2.3 Alternative Cloud Platforms Considered

Multiple cloud platform alternatives were evaluated on scalability, availability of managed services, IaC support, ecosystem maturity, operational complexity, and compatibility with the existing baseline. The main alternatives were AWS, Microsoft Azure, Google Cloud Platform (GCP), continuation on DigitalOcean, and self-managed on-premises infrastructure.

AWS was the strongest candidate due to the maturity of its managed-service ecosystem and its fit with the migration objectives. Services such as ECS Fargate, RDS PostgreSQL, S3, Cognito, SES, and CloudFront natively supported the target model, with strong Terraform integration and operational scalability aligned with the company's cloudification strategy; existing operational knowledge was also partially AWS-aligned. **Azure** offered a broad enterprise ecosystem with equivalent services and strong governance capabilities, but introduced additional migration complexity because the existing direction was more closely aligned with AWS. **GCP** was considered mainly for its Kubernetes-native strengths but offered less alignment with the established architectural direction. **Continuation on DigitalOcean** would have minimized short-term effort but lacked the managed-service integration and automation of AWS. **On-premises** infrastructure offered direct control but significantly increased operational responsibilities, conflicting with the goal of reducing operational burden.

Table 6.1. Comparative assessment of cloud provider alternatives for Outsight Cloud migration

Criterion	**AWS**	**Azure**	**GCP**	**On-prem**
Managed services fit	High	Medium	Medium	Low
IaC and automation	High	Medium	Medium	Low
Operational complexity	Low to medium	Medium	Medium	High
Ecosystem and maturity	Very high	High	High	Variable
Migration continuity with current baseline	High	Medium-low	Medium-low	Low

6.2.4 Data-Layer and Identity Options

Given the limitations of the legacy datastore identified in Chapter 5, the evaluation focused on *how* to reach the target relational model, not on whether to adopt PostgreSQL. An immediate cutover simplified the long-term architecture but introduced high migration risk; a long-term dual-backend without a clear endpoint was operationally safer but risked prolonging dependency concentration. The selected approach relied on progressive validation and controlled switchover, allowing data-transfer correctness, API behavior, and rollback readiness to be verified before the final transition.

Identity modernization was evaluated similarly. Postponing IAM modernization was simpler but risked extending coupling to legacy authentication; a complete redesign from the start was architecturally cleaner but introduced excessive risk. The strategy therefore favored progressive IAM modernization based on compatibility-preserving changes and staged adoption of standards-based flows.

6.3 Technical and Economic Evaluation Criteria

Each candidate was evaluated against common criteria. Technical evaluation emphasized backward compatibility with existing workflows and service behaviors, security and readiness for standards-based IAM, deployment reproducibility (versioned

definitions, automated provisioning, consistent environments), migration testability (incremental validation, rollback readiness, automated verification), and operational observability. Economic and operational evaluation considered recurring infrastructure costs, reduction of operational overhead and manual workflows, dependency concentration around high-cost external services, and maintainability and automation capabilities.

6.4 Selected Strategy and Rationale

The selected strategy was an incremental migration centered on managed AWS services, Infrastructure as Code, and independently verifiable migration steps. AWS-managed services were adopted for runtime execution, routing, storage, authentication, and observability, with ECS Fargate selected over Kubernetes (Section 6.2.2), and Terraform as the primary infrastructure control layer for reproducible provisioning and environment governance. For the data layer, the strategy applied the controlled transition model of Section 6.2.4, building on the PostgreSQL target of Chapter 5.

AWS was selected because it provided the most complete alignment with the technical and operational requirements: mature Terraform integration, strong scalability, managed services aligned with the target architecture (including RDS PostgreSQL), developer-friendly customizability (e.g. Lambda-based logic), and continuity with the company's cloudification strategy. The main acknowledged trade-off is that AWS can become expensive under specific scaling conditions, but for this project the balance between service quality, customization, and migration continuity remained favorable. The rationale was to make each architectural change independently verifiable, so modernization could proceed through controlled increments rather than a single high-risk event.

6.5 Risk Management and Coexistence Plan

Risk management was integrated directly into the migration strategy. Migration steps were designed to remain independently verifiable through validation workflows, data-consistency checks, and coexistence-oriented testing, with development

and production environments managed separately. Special attention was given to dual-backend coexistence during data-layer migration, with validation workflows introduced to detect behavioral divergence early and support controlled switchover decisions. Coexistence was thus treated not as an informal transition phase but as a structured engineering stage. This reflects Morris's principle of reducing the scope of each change: smaller increments limit the blast radius of any failure and make problems easier to detect and correct [2].

Chapter 7

Target Architecture and Solution Design (To-Be)

This chapter presents the target architecture resulting from the decision framework in Chapter 6. The baseline is described in Chapter 5; here the focus is on the migration destination designed for incremental adoption.

7.1 Architecture Overview

The migration of Outsight Cloud to AWS required a cloud-native architecture able to improve scalability, maintainability, and service integration while preserving the original application workflow. The selected design follows a layered model with four levels: a client and delivery layer, a load balancing and routing layer, an application layer, and a managed services layer. The resulting left-to-right request flow starts from the browser, continues through CloudFront and an Application Load Balancer, reaches frontend and backend services on ECS Fargate, and integrates with managed services including PostgreSQL, Redis, Amazon S3, Cognito, SES, and SNS (Figure 7.1).

7.2 Client, Delivery, and Routing Layers

The browser is the user entry point to Outsight Cloud, used for simulation visualization, organization and site management, document access, and file operations,

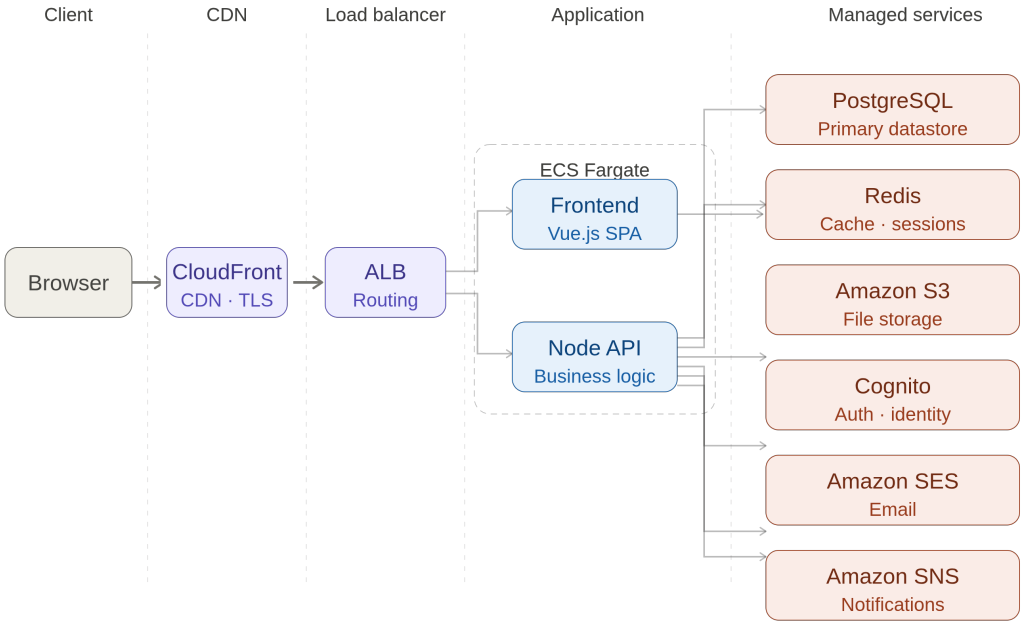

Figure 7.1. Final AWS architecture adopted for Outsight Cloud.

all over HTTPS. **Amazon CloudFront** acts as the content delivery layer between users and the internal infrastructure, reducing latency, caching static assets, and reducing backend load, which is particularly relevant for the Vue frontend static resources. The **Application Load Balancer** (ALB) is the central routing component: it receives requests forwarded by CloudFront and distributes traffic across ECS Fargate services, providing request routing, health checks, HTTPS termination, and traffic distribution, enabling independent scaling and improved fault tolerance.

7.3 Application Layer

The application layer runs on **Amazon ECS Fargate** with a serverless container execution model that removes the need to provision, patch, and scale virtual machines manually. This choice follows the evaluation in Chapter 6: Kubernetes was not selected because the associated cluster-management overhead was not justified by the scale of Outsight Cloud during the migration. The **frontend service** (Vue.js) renders the user interface, handles interactions, and manages authentication flows, while the **backend service** (Node.js) exposes the core APIs orchestrating business logic for organizations, sites, simulations, authentication, file operations, notifications, and integrations with AWS services. Deploying each as a dedicated container enables independent release and scaling.

7.4 Managed Services Layer

PostgreSQL replaced Airtable as primary data storage, moving from a semi-structured backend toward a relational model with stronger consistency guarantees; core entities such as users, organizations, sites, and simulations are persisted there. **Redis** is used as in-memory cache and session storage to reduce persistent-storage accesses and improve response time. **Amazon S3** provides object storage for simulation files, uploaded assets, and documentation, selected for scalability, durability, and native AWS integration. **Amazon Cognito** handles authentication and identity management (registration, login, verification, token handling), reducing custom authentication logic. **Amazon SES** handles outgoing email (invitations, account messages, notifications), and **Amazon SNS** supports asynchronous, event-driven

notifications for decoupled communication patterns.

7.5 Network and Security Topology

The AWS deployment was designed around a simplified cloud-native networking model focused on incremental migration and controlled exposure of public services. The infrastructure operates inside an AWS Virtual Private Cloud (VPC) with multiple public subnets across Availability Zones to improve availability. ECS Fargate services run with public outbound connectivity to simplify communication with managed services and reduce networking overhead during early stages. Despite the use of public subnets, direct external access to containers is restricted through dedicated Security Groups: the ALB accepts inbound HTTP/HTTPS traffic from external clients, ECS services accept traffic only from the ALB, and containers are therefore not directly exposed to the Internet. TLS is managed through AWS Certificate Manager (ACM) for centralized certificate lifecycle management. The architecture also separates application traffic from static asset delivery: simulation files are stored in private S3 buckets distributed through CloudFront, with access restricted through CloudFront Origin Access Control (OAC). Overall, the topology reflects a pragmatic migration-oriented design balancing security and operational simplicity while preserving the possibility of future evolution toward more isolated network architectures.

7.6 Architectural Benefits

Compared to the legacy architecture of Chapter 5, the target design directly addresses the migration drivers of Chapter 1 and the decision criteria of Chapter 6. First, it provides clearer separation of concerns between presentation, application services, data storage, and supporting services, reducing coupling and simplifying future evolution. Second, the adoption of managed services significantly reduces operational responsibility, delegating load balancing, TLS management, storage scalability, and database availability to AWS. Third, it improves scalability: frontend and backend scale independently through ECS Fargate, and managed storage and

database services evolve independently from application deployments. From a maintainability standpoint, Infrastructure as Code through Terraform establishes a reproducible, version-controlled infrastructure model, reducing configuration drift. The migration from Airtable to PostgreSQL further strengthens ownership of the data layer through explicit relational schemas and database-level constraints. Finally, environment separation, automated provisioning, and the validation mechanisms described in Chapter 8 provide a stronger foundation for controlled deployments and incremental evolution. Overall, the final AWS architecture is not simply a replacement hosting environment but a cloud-native operational model designed for long-term sustainability.

Chapter 8

Migration Implementation

This chapter describes how the migration strategy was executed in practice, covering the infrastructure transition, data-layer migration, and validation workflows that enabled the controlled move from the legacy baseline to the target AWS architecture.

8.1 Incremental Execution Plan

The migration was implemented incrementally, structured around four tracks executed in a deliberate sequence: establishing cloud infrastructure on AWS; formalizing reproducible provisioning with Terraform; migrating data from Airtable to PostgreSQL; and validating migrated data, application behavior, and deployment correctness before production switchover. This sequencing made the migration technically traceable, evaluating infrastructure readiness, provisioning reproducibility, data-transfer correctness, and application validation as separate workstreams and reducing the risk of cascading failures. The infrastructure migration was executed first, allowing AWS environments, networking, load balancing, and deployment workflows to be validated independently before the data transition. Figure 8.1 summarizes this execution model.

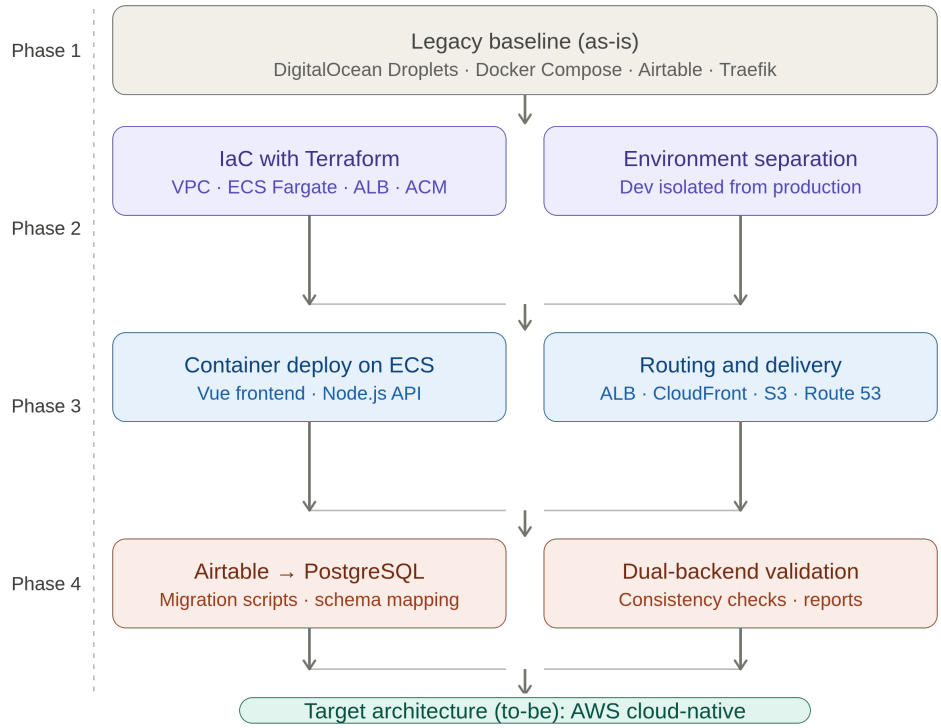

Figure 8.1. Incremental execution flow from the legacy baseline to the target AWS cloud-native architecture

8.2 Infrastructure as Code and Environment Separation

Infrastructure as Code (IaC) is a software engineering approach in which infrastructure is provisioned and managed through code instead of manual procedures [7]. Infrastructure definitions become declarative artifacts stored in version control, replacing fragile practices based on manual console operations or ad-hoc scripts and improving consistency and reliability [7]. Morris identifies three mutually reinforcing core practices: defining everything as code, continuously testing and delivering all work in progress, and building small loosely coupled pieces [2]. A key characteristic is that engineers define the desired final state while automation computes

and executes the required operations, providing consistency across environments, auditable version control, automation, reproducibility, and maintainability. In the Outsight Cloud migration, IaC was adopted to automate AWS provisioning and reduce manual intervention during environment setup and updates.

8.2.1 Terraform-Driven Provisioning

Terraform is an IaC tool by HashiCorp that enables provisioning and lifecycle management of infrastructure through declarative configuration files (HCL) that can be versioned, reviewed, and integrated into delivery workflows [8]. It interacts with platforms through providers; in this migration, the AWS provider managed ECS Fargate services, Application Load Balancers, Route 53 records, ACM certificates, VPC networking, S3 storage, and observability infrastructure.

Listing 8.1. Simplified ECS Fargate service definition

```
resource "aws_ecs_service" "backend" {
  name = "outsight-backend"
  cluster = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.backend.arn

  desired_count = 1
  launch_type = "FARGATE"

  network_configuration {
    subnets = aws_subnet.public[*].id
    security_groups = [aws_security_group.ecs.id]
    assign_public_ip = true
  }
}
```

Listing 8.1 shows a simplified resource definition used to deploy a backend container as an ECS Fargate service: through declarative configuration, containerized workloads previously orchestrated with Docker Compose could be provisioned reproducibly in AWS while preserving the same packaging model. The Terraform workflow follows three phases [8]: *write* (define the target state declaratively), *plan* (compare desired and current state, showing which resources will be created, updated, or

removed), and *apply* (execute the planned operations respecting resource dependencies). This workflow was especially useful for migration safety, since planned changes could be reviewed before execution and applied incrementally. Terraform also keeps a state representation of real infrastructure (`terraform.tfstate`) to compute incremental changes and detect drift [8]. Terraform was selected because it aligned with the migration objectives: versioned infrastructure definitions, automation integrated into CI/CD processes, reproducible environments, incremental changes, and modular maintainable definitions.

A dedicated effort was made to separate development and production concerns through environment-specific configuration and controlled evolution of shared resources. This separation was particularly important during migration because it allowed AWS environments to be validated independently before production workloads were progressively redirected to the new platform.

8.3 Infrastructure Migration Execution

While Chapters 5 and 7 describe the legacy platform and target design, this section documents how the transition was executed. The objective was to replace the previous Docker Compose and Traefik deployment model with a reproducible cloud-native foundation without forcing a simultaneous application and data cutover.

The first step translated the target architecture into an IaC baseline using Terraform, provisioning the main building blocks of the target platform: ECS Fargate services for frontend and backend containers, an Application Load Balancer for public routing, Route 53 records for DNS, ACM certificates for HTTPS, CloudWatch log groups for centralized logging, and S3 together with CloudFront for simulation file storage and delivery. A key difference concerned request routing: whereas the legacy environment used Traefik as reverse proxy and HTTPS entry point, routing responsibilities were transferred to the ALB and defined through listener rules mapping traffic to target groups. The backend service was adapted so that its externally exposed routes aligned with the ALB routing model, with a dedicated health-check endpoint for service probes. Container images were published through the existing private registry, and registry credentials were supplied at runtime through AWS Secrets Manager rather than embedded in IaC artifacts or Terraform state.

Simulation file storage was redesigned around a private S3 bucket fronted by CloudFront, replacing the previous S3-compatible integration with an AWS-native delivery model while keeping the bucket non-public through CloudFront Origin Access Control. Public HTTPS access was enabled through ACM-managed certificates integrated with the ALB and CloudFront, and Route 53 managed DNS records per environment. Once provisioned, validation focused on verifying that the new infrastructure behaved correctly before redirecting production traffic, covering ECS service startup, ALB routing, HTTPS and DNS configuration, centralized logging, and the S3/CloudFront flow. Development and production concerns were kept separated so that experimental changes and database modernization could be isolated from the production system still running on the legacy platform. The final transition consisted of progressively redirecting public access from the legacy environment to AWS through DNS updates in Route 53, avoiding a single disruptive cutover across infrastructure, routing, and data storage simultaneously.

8.4 Data Migration Tooling and Validation

The migration path from Airtable to PostgreSQL was implemented with schema and migration tooling designed to preserve compatibility constraints, aiming not only at data transfer but at controlled evolution toward explicit schema ownership. Dedicated scripts were developed to replicate data from Airtable into PostgreSQL, validate consistency through machine-readable reports, and verify migration correctness at each checkpoint before production switchover. These artifacts transformed data migration from a one-time operation into a repeatable and verifiable workflow, providing measurable confidence in the correctness of the transition before the final cutover.

Chapter 9

Evaluation and Discussion

The evaluation in this chapter is grounded in a reproducible, script-driven benchmark framework developed for this report, complemented by a cost and operational-effort analysis. Measurements are collected through automated HTTP requests against both the legacy and the new AWS environment, stored in structured CSV files, and post-processed with Python.

9.1 Benchmark Methodology and Demand Profile

The benchmark targets the Outsight Cloud web application at the HTTP layer. For each environment, five timing components are extracted from every request using `curl`'s built-in timing variables, decomposing the total round-trip time into distinct network and application phases. Table 9.1 lists each metric and its definition.

The latency benchmark focuses on the homepage endpoint (`GET /`), which is publicly accessible and exercises the full network stack (DNS resolution, TCP connection, TLS handshake, and application response), making it the most informative single target for an end-to-end comparison. Authenticated API endpoints and static asset downloads were not included, as they require a stable authentication token and a consistent asset fingerprint across both environments.

Each benchmark run consists of $N = 100$ independent HTTP GET requests issued sequentially, spaced to avoid connection reuse: each `curl` invocation opens

Table 9.1. HTTP timing metrics extracted from `curl` and their meaning. All values are recorded in seconds by `curl` and converted to milliseconds during post-processing.

Metric	**`curl` variable**	**Description**
DNS Lookup	`time_namelookup`	Elapsed time from request start until DNS name resolution completes.
TCP Connect	`time_connect`	Elapsed time from request start until the TCP three-way handshake completes.
TLS Handshake	`time_appconnect`	Elapsed time from request start until the TLS/SSL handshake and key exchange complete.
TTFB	`time_starttransfer`	Time to First Byte: from request start until the first response byte is received.
Total Time	`time_total`	Total elapsed time of the complete HTTP transaction, including response body transfer.

a fresh TCP connection and performs a complete TLS handshake, so no warm-up effect inflates the results. Table 9.2 summarises the configuration parameters. With 100 samples, the 95 % confidence interval for the mean (Student t-distribution) has a half-width of about 3.4 ms for TTFB under the legacy environment, a relative margin below 3 %. The benchmark also reports the 90th, 95th, and 99th percentiles, which are most relevant to service-level objectives; outliers are deliberately not removed, preserving real-world tail behaviour.

Table 9.2. Benchmark configuration parameters used for all latency measurements.

Parameter	**Value / Description**
Sample size per environment	$N = 100$ independent requests
Request timeout	30 s (curl `-max-time`)
Retry policy	1 automatic retry on transport error
HTTP method	GET
Protocol	HTTPS (TLS 1.2 / 1.3)
Confidence level	95 % (Student t-distribution)
Percentiles reported	P90, P95, P99
Measurement tool	`curl` built-in timing variables
Output format	CSV (one row per request)
Client location	Fixed single location (Paris, FR)

The two benchmark targets expose the same application over HTTPS but differ substantially in their underlying infrastructure (Table 9.3). To ensure a fair comparison, all 200 measurements were issued from the same client machine in Paris, France, within the same time window and with no client-side caching, so the only variable between the two series is the target base URL.

9.2 Cost and TCO Analysis

This section quantifies the economic impact of the migration by comparing the total monthly spend of the legacy platform with that of the new AWS architecture. AWS costs are estimated from the current daily run rate extrapolated to a monthly figure, excluding transient migration overhead; all figures are in EUR.

The legacy platform relied on three services: DigitalOcean Spaces (**€70/month**) for object storage, DigitalOcean Droplets (**€102/month**) hosting the application

Table 9.3. Infrastructure comparison between the As-Is (Legacy) and To-Be (AWS) benchmark targets. Both environments expose the same application over HTTPS.

Aspect	**As-Is**	**To-Be**
Hosting platform	DigitalOcean Droplet	AWS ECS Fargate
Container orchestration	None	ECS with auto-scaling task definitions
CDN / edge caching	None	Amazon CloudFront
DNS resolver	Standard recursive DNS	Amazon Route 53
TLS termination	Nginx + Let's Encrypt	CloudFront + ACM certificate
Load balancer	Nginx reverse proxy	AWS Application Load Balancer
Deployment region	Amsterdam	Paris
Measured base URL	`legacy.cloud.outsight.ai`	`cloud.outsight.ai`

stack, and Airtable with 7 editor licences (**€294/month**) as the primary operational database. The total legacy monthly spend was therefore **€466/month**.

The AWS platform distributes the workload across managed services. Figure 9.1 shows the AWS cost breakdown by service, and Table 9.4 maps each category to the corresponding legacy and AWS service. The three largest drivers are ECS Fargate (€59/month), Elastic Load Balancing (€53/month), and Amazon RDS (€48/month); RDS replaces Airtable as the primary data store while providing full SQL capabilities, automated backups, and high-availability failover at a fraction of the Airtable licence cost.

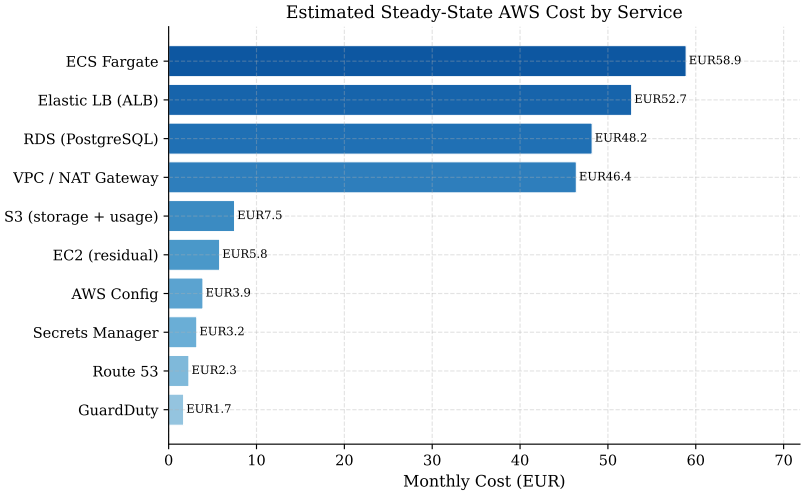

Figure 9.1. Estimated steady-state AWS cost by service (EUR/month).

Compared to the legacy total of €466/month, the AWS platform reduces the monthly spend to **€231/month**, a saving of **€235/month (–50 %)**, primarily driven by the elimination of the Airtable licence (€294/month), replaced by RDS at €48/month. Figure 9.2 presents the comparison by category: the Database/SaaS category drops by 84 % and storage by 89 % (S3 at €8/month versus €70/month for DigitalOcean Spaces), while three categories absent from the legacy architecture (Load Balancer, Networking/VPC, and Security) now appear as distinct lines. The saving is structural: it stems from eliminating a SaaS tool whose per-seat pricing was ill-suited to an infrastructure role, and replacing it with a managed database priced on actual resource consumption. The operational effort freed by the transition (Section 9.4) provides additional indirect savings.

Table 9.4. Monthly cost comparison between the Legacy and AWS platforms.

Category	**Legacy**	**EUR/mo**	**AWS**	**EUR/mo**
Storage	DigitalOcean Spaces	70	Amazon S3	7.5
Compute	DigitalOcean Droplets	102	ECS Fargate	58.9
Database / SaaS	Airtable (7 editors)	294	Amazon RDS	48.2
Load Balancer	–	–	Elastic LB (ALB)	52.7
Networking	–	–	VPC + Route 53	48.7
Security	–	–	GuardDuty + Secrets Mgr	4.9
Governance	–	–	AWS Config	3.9
Total		**466**		**224.8**

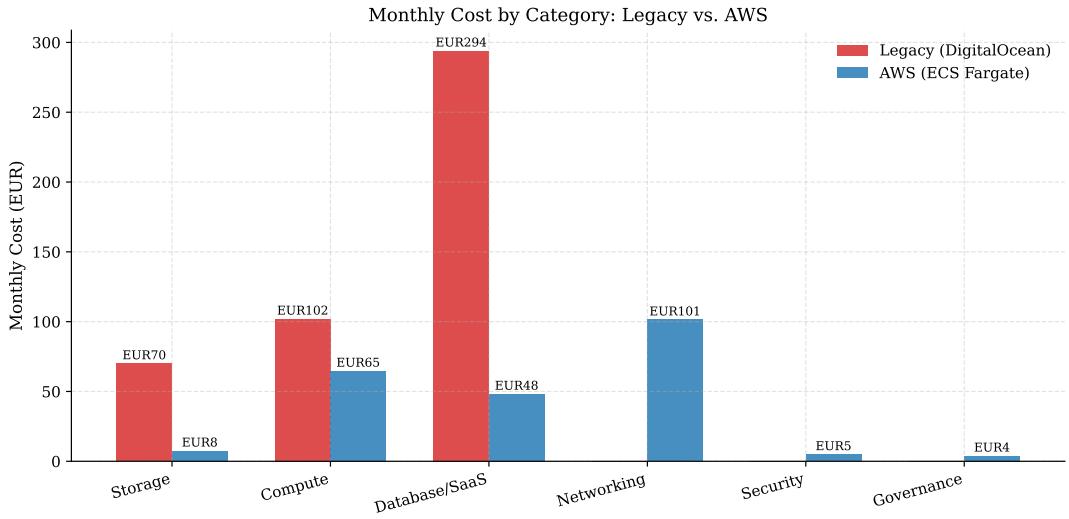

Figure 9.2. Monthly cost by category: Legacy vs. AWS (EUR/month). Categories absent from the legacy platform are shown as zero.

9.3 Latency and Performance Analysis

To complement the cost analysis, an HTTP-level latency benchmark quantified the end-to-end performance improvement. The benchmark measures five timing components (DNS resolution, TCP connection, TLS handshake, Time to First Byte, and Total request time) over $N = 100$ requests per environment, with a 95 % confidence interval from Student's t-distribution. Table 9.5 reports the complete descriptive statistics.

Under the legacy architecture, every timing component exhibits both a high mean and significant variance. DNS resolution averages **14.92 ms** with a 99th-percentile of 60.70 ms, revealing substantial jitter attributable to the absence of a dedicated resolver and anycast routing. TCP connection time mirrors the DNS distribution (mean 15.19 ms). The TLS handshake is the dominant contributor, averaging **84.83 ms** (max 333.05 ms), a consequence of a single-region droplet with no session-resumption optimisation. The aggregate TTFB reaches a mean of **130.74 ms** (p95 = 177.00 ms, max = 394.91 ms), meaning some users routinely experience response times more than three times the minimum observed value.

The AWS architecture, built on ECS Fargate behind an ALB with CloudFront

Table 9.5. Descriptive statistics for the Home endpoint benchmark ($N = 100$ requests per environment). All timing values in milliseconds (ms).

Environment	**Metric**	**Mean**	**Median**	**Std Dev**	**Max**	**P90**	**P95**	**P99**
Legacy (DigitalOcean)	DNS Lookup	14.92	7.58	15.38	114.92	30.01	36.39	60.70
	TCP Connect	15.19	7.81	15.45	115.19	30.32	37.47	60.99
	TLS Handshake	84.83	81.40	31.20	333.05	104.09	118.50	169.66
	TTFB	130.74	124.85	34.08	394.91	156.69	177.00	211.58
	Total Time	130.83	124.94	34.09	395.02	156.80	177.13	211.71
AWS (ECS Fargate)	DNS Lookup	1.64	1.15	2.20	22.51	2.43	2.49	2.92
	TCP Connect	1.79	1.27	2.23	22.74	2.67	2.78	3.25
	TLS Handshake	44.45	43.30	13.82	146.38	55.53	58.90	71.90
	TTFB	81.09	78.59	19.73	197.31	100.82	105.22	149.57
	Total Time	81.16	78.64	19.75	197.43	100.92	105.29	149.69

as the global CDN, dramatically reduces every component. DNS resolution drops to a mean of **1.64 ms** (nearly two orders of magnitude lower), thanks to Route 53's anycast infrastructure and short CloudFront TTLs, with the standard deviation falling ninefold. TCP connection time averages **1.79 ms**, and the TLS handshake settles at **44.45 ms**, benefiting from CloudFront's TLS 1.3 support and session-ticket resumption. Most importantly, TTFB falls to **81.09 ms** on average (p95 = 105.22 ms), with the CI width narrowing to 7.83 ms, indicating a more consistent experience.

Figure 9.3 presents the mean latency per component with 95 % CI error bars and percentage improvement, ranging from **−89%** on DNS and TCP to **−38%** on TTFB and Total time. Even the more modest TLS (−48%) and TTFB (−38%) reductions translate to absolute savings of 40 ms and 50 ms on every request.

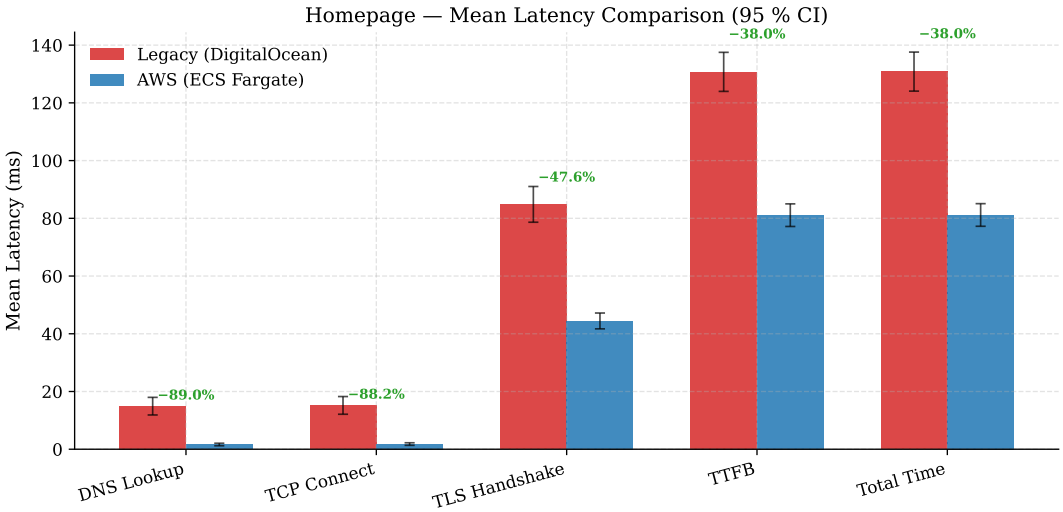

Figure 9.3. Mean latency comparison, Legacy vs. AWS, per HTTP timing component, with 95% CI error bars.

Figures 9.4 and 9.5 complement the mean comparison by showing the full distributions. The legacy boxes span a much wider interquartile range with numerous high-value outliers, whereas the AWS boxes are compact and nearly outlier-free for DNS and TCP. The ECDFs confirm the tail behaviour: for TTFB, the legacy distribution exhibits a long right tail beyond 350 ms, whereas 99% of AWS requests complete within 150 ms, and the AWS p95 TTFB (105 ms) is lower than the legacy *median* (125 ms).

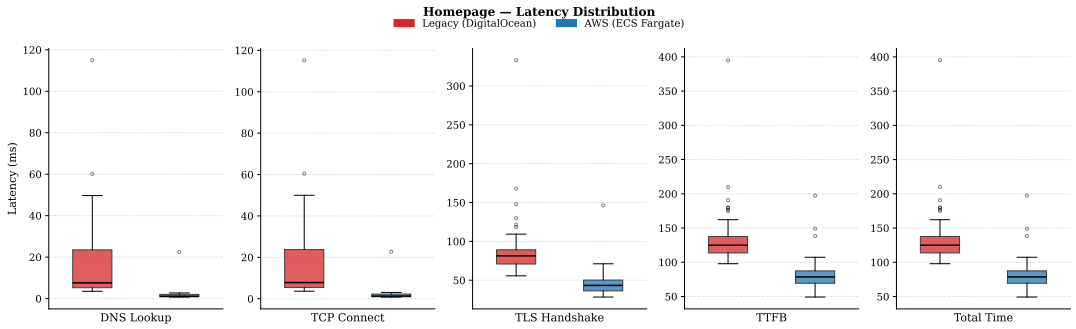

Figure 9.4. Box plots of latency distributions per HTTP timing component (100 samples per environment).

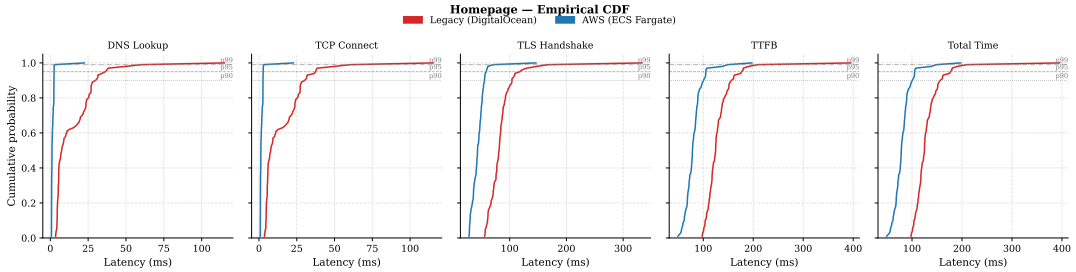

Figure 9.5. ECDFs of latency per HTTP timing component, with p90/p95/p99 reference lines.

In summary, the migration reduces mean end-to-end latency by approximately 38% while tightening the distribution and eliminating the long tail observed on the legacy platform. The gains are most pronounced in the network-proximity-sensitive phases (DNS, TCP), where Route 53, CloudFront, and the AWS backbone replace an unoptimised single-region setup. The residual TLS and TTFB latencies are now bounded by application processing time and the physical distance between the AWS region and the client, both architectural lower bounds rather than avoidable inefficiencies.

9.4 Operational Cost and Efficiency

Beyond direct infrastructure spending, the architecture of a platform determines the engineering time required to keep it running. This section analyses the recurring operational tasks associated with each architecture and estimates their cost in

engineering hours.

The legacy platform, built on DigitalOcean Droplets running Docker Compose, placed the full burden of infrastructure operations on the team. *OS and runtime maintenance* (manual security patches, kernel and Docker Engine upgrades, coordinated across machines) consumed about 4–6 hours per month. *Deployment and rollback* were performed by SSH-ing into the droplet and restarting services with Docker Compose, a procedure that lacked atomicity and carried brief service interruptions, estimated at 3–4 hours per deployment cycle. *Scaling and capacity management* required manually resizing or duplicating droplets with no automatic scaling policy, adding 2–3 hours per month. *Monitoring and incident response* relied on ad-hoc log collection via SSH with no centralised observability, adding 1–2 hours per incident. The cumulative recurring effort amounted to an estimated 10–15 hours per month of undifferentiated infrastructure maintenance; at a loaded cost of €80–100/hour, this corresponds to a recurring overhead of €800–1 500/month adding no direct product value.

The AWS architecture delegates most of these tasks to managed services, changing the team's role from infrastructure operator to configuration manager. ECS Fargate is serverless, so there is no OS to patch, no Docker Engine to maintain, and no kernel to update, reducing OS and runtime maintenance to effectively zero. ECS rolling deployments update tasks incrementally with zero downtime, are triggered through the CI/CD pipeline, and support instantaneous rollback by re-registering a previous task-definition revision. ECS Service Auto Scaling adjusts the number of running tasks automatically, replacing reactive capacity planning with policy configuration. Amazon CloudWatch collects logs from ECS tasks, ALB access logs, and RDS performance insights in a single pane, with dashboards and alarms configured as code, while Amazon RDS automates backups (point-in-time recovery up to 35 days), minor version upgrades, storage auto-scaling, and failover. The residual recurring operational effort is estimated at 2–3 hours per month, covering configuration updates, cost monitoring, IAM policy reviews, and pipeline maintenance.

Table 9.6 summarises the comparison. At a conservative loaded rate of €80/hour and using the midpoint of each range, the legacy platform consumed approximately **€1 320/month** in operational effort, reduced to approximately **€140/month** in the AWS architecture, a saving of **€1 180/month**. Combined with the €235/month

saving on direct infrastructure costs (Section 9.2), the migration yields a total estimated saving of **€1 415/month**, roughly **€17 000** per year, without accounting for the reduction in incident frequency enabled by centralised observability.

Table 9.6. Operational effort comparison between the Legacy and AWS architectures. Estimates are in engineering hours per month.

Activity	**Legacy (h/mo)**	**AWS (h/mo)**
OS and runtime patching	4–6	0
Deployment and rollback	3–4	0.5
Scaling and capacity mgmt	2–3	0
Monitoring and log management	2–4	0.5
Database administration	2–3	0.5
Incident investigation overhead	1–2/incident	minimal
Total (excl. incidents)	**13–20**	**1.5–2**

The operational figures are informed engineering estimates rather than precisely measured timesheet data. To assess whether the *magnitude* of the reduction is plausible, it is useful to compare it against independent industry evidence. A Total Economic Impact study by Forrester Consulting commissioned by AWS reports, among other findings, provisioning time savings of about 97.5 % through automated environment creation, a 30 % improvement in DevOps productivity, and a 90 % reduction in high-priority incidents after adopting managed services and IaC [9]. Although that study refers to a large enterprise and a broad AWS portfolio rather than the specific ECS Fargate, RDS, and Terraform stack used here, the underlying mechanisms are the same, and the order-of-magnitude reduction estimated for Outsight Cloud (from 13–20 to 1.5–2 hours per month, roughly 90 %) is consistent with these figures. For transparency, the Forrester study is vendor-commissioned and based on a composite organization, so its figures are used only as an external plausibility check on the direction and magnitude of the savings, not as a direct measurement of Outsight Cloud.

Chapter 10

Conclusion

This report investigated the modernization of Outsight Cloud through the migration of a production platform from a legacy infrastructure toward a cloud-native architecture on AWS. The original system relied on self-managed infrastructure and Docker Compose while depending on several external services, most notably Airtable as the primary operational datastore. Although this enabled rapid early development, it progressively introduced limitations related to scalability, maintainability, operational governance, and dependency management. Rather than approaching migration as a simple infrastructure relocation, this work treated cloud adoption as a broader architectural transformation involving infrastructure modernization, data-layer evolution, deployment automation, and operational redesign, supported by concrete engineering artifacts (Terraform definitions, data migration tooling, validation scripts, and environment-specific deployment workflows).

10.1 Key Outcomes

The evaluation in Chapter 9 provides quantitative evidence across three dimensions. From a cost perspective (Section 9.2), the migration delivers a structural saving of €235/month driven primarily by replacing Airtable with Amazon RDS, eliminating a per-seat SaaS pricing model ill-suited to a backend role, reinforced by reduced object-storage costs. From an operational standpoint (Section 9.4), delegating patching, scaling, deployment, and database administration to managed services reduces the team's undifferentiated operational burden by roughly 90 %, both in hours and cost.

From a performance perspective, the HTTP benchmark shows consistent improvements across all timing components, most pronounced in the DNS and TCP phases, with a tighter latency distribution that brings worst-case user experience closer to the median.

Beyond the measured outcomes, two structural contributions complete the picture. The adoption of Terraform established a reproducible, version-controlled, and auditable provisioning baseline, re-establishing ownership over infrastructure and data assets. The incremental migration design, supported by dual-backend coexistence, data replication tooling, validation scripts, and environment separation, minimized migration risk and service disruption while establishing a foundation of independently scalable managed services capable of supporting long-term platform evolution.

10.2 Limitations and Open Questions

Two aspects of the measurement scope should be acknowledged. The latency benchmark targets the publicly accessible homepage endpoint, which provides a reproducible, infrastructure-representative baseline; authenticated API endpoints and database-intensive workflows were not included, as their evaluation requires a stable authentication token and a consistent asset fingerprint across both environments. The operational effort figures are based on informed engineering estimates grounded in direct team experience rather than systematically recorded timesheet data.

10.3 Future Work

Several directions remain open. The benchmark coverage could be extended to authenticated API endpoints and database query performance, complementing the infrastructure-level measurements with application-layer evidence. Load testing under concurrent demand would confirm whether ECS Fargate auto-scaling and RDS connection pooling perform as expected under peak conditions. The identity and access management layer introduced through Amazon Cognito could be further standardized, and observability could be extended through richer CloudWatch

dashboards and post-cutover performance baselines. Finally, the migration principles documented here (the decision framework, IaC-first provisioning, dual-backend validation, and incremental rollout planning) could be evaluated in other industrial contexts to assess their generalizability.

Overall, this report demonstrates that legacy-to-cloud-native migration is most effective when treated as a structured engineering process supported by explicit architectural decisions, reproducible infrastructure management, continuous validation, and controlled incremental evolution. The Outsight Cloud case study shows how modernization can be grounded in concrete engineering practices, and the quantitative evaluation confirms measurable improvements across cost, latency, and operational overhead, validating the architectural decisions made throughout the process.

Bibliography / References

[1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media, 2021.

[2] K. Morris, *Infrastructure as Code: Dynamic Systems for the Cloud Age*, 2nd ed. O'Reilly Media, 2021.

[3] Amazon Web Services. «What is cloud migration?» Accessed: 2026-05-17. (2026), [Online]. Available: `https://aws.amazon.com/what-is/cloud-migration/`.

[4] Amazon Web Services. «What is iam?» Accessed: 2026-05-31. (2026), [Online]. Available: `https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html`.

[5] Airtable. «Introduction to airtable basics». Accessed: 2026-05-25. (2026), [Online]. Available: `https://support.airtable.com/docs/introduction-to-airtable-basics`.

[6] The Kubernetes Authors. «Overview of kubernetes». Accessed: 2026-05-30. (2026), [Online]. Available: `https://kubernetes.io/docs/concepts/overview/`.

[7] Amazon Web Services. «Infrastructure as code». Accessed: 2026-05-17. (2026), [Online]. Available: `https://docs.aws.amazon.com/whitepapers/latest/introduction-devops-aws/infrastructure-as-code.html`.

[8] HashiCorp. «What is terraform?» Accessed: 2026-05-17. (2026), [Online]. Available: `https://developer.hashicorp.com/terraform/intro`.

[9] Forrester Consulting, «The total economic impact of AWS cloud operations: Cost savings and business benefits enabled by operating on AWS», Forrester Research, Total Economic Impact Study commissioned by AWS, May 2022.